A Systematic Benchmark of Supervised and Unsupervised Graph Embedding Methods

Konstantina Koulaktsidou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Fragiska Kiminou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Nikolaos Moraitis
Department of Electrical and Computer Engineering
National Technical University of Athens
email

***Abstract*—Graph embedding techniques are important tools for representing graphs in low-dimensional vector spaces, enabling effective downstream tasks. In this work, we evaluate unsupervised and supervised representative whole graph embedding techniques on real world benchmark datasets. Specifically, we compare Graph2Vec, NetLSD, and the Graph Isomorphism Network (GIN) across three datasets (MUTAG, ENZYMES, and IMDB-MULTI). Performance is assessed through graph classification and clustering tasks, considering predictive accuracy, computational efficiency and scalability. Our expertiments indicate that GIN achieves the highest accuracy but requires substatial computational resources. Graph2Vec balances performance and efficiency, while NetLSD provides competitive clustering performance with minimal overhead. Therefore it is apparent that each method has its distinct trade-offs. All methods maintain reasonable robustness under moderate perturbations, with sensitivity depending on dataset characterestics. These findings provide practical insights that aid in selecting graph embedding approaches based on the task requirements, dataset properties and computationional constraints.**

***Index Terms*—Graph2Vec, NetLSD, GIN, Graph Embedding, Graph Classification, Graph Clustering**

I. Introduction

Graphs are known to provide a powerful representation for complex relational data and are widely used in domains such as network analysis, bioinformatics and chemistry. In many scenarios, the learning task is defined at the graph level, such as classifying molecular graphs by biological activity. However, graphs have irregular structure with different variable sizes, unordered nodes and complex connectivity patterns, features that make them face challenges to traditional machine learning algorithms, which typically work with fixed dimensional vector inputs.

Graph embedding methods address this challenge by mapping entire graphs into continuous vector representations that preserve the structural and semantic information of the graph and this helps the ML techniques to work efficiently to downstream tasks, such as classification or clustering and even similarity search. As a result, graph embeddings have successfully become a core component for current graph based learning pipelines.

Early approaches relied on handcrafted features or graph kernels, which often suffer from high computational complexity or limited expressiveness [6], [7]. Now recent approaches automatically learn graph representations and can be categorized into unsupervised [1], [2] and supervised techniques [3]. Unsupervised methods learn graph representations without relying on labeled data exploiting often structural patterns [1], [2]. On the other hand, supervised procedures [3] are based on graph neural networks (GNNS) [8] and learn task specific embeddings through end-to-end training. However, choosing this type of procedure typically comes with the cost of increased computational complexity, higher memory consumption and the need for labeled training data, which is often not provided to us.

Despite the development of graph embedding techniques [1], [2], [3], comparisons between unsupervised and supervised methods remain limited [9], [10]. Many existing studies focus primarly on classification accuracy and overlooking significant considerations such as computational cost, embedding dimensionality and robustness to structural permutations [9]. These are critical when working with real world small datasets (like EMB1 category), where overfitting, instability and minimum training data can easily affect the model's performance.

In this work, we focus on EMB1 and we evaluate and compare unsupervised and supervised graph embedding methods on small benchmark datasets from the TU-Dataset collection. Studying small datasets is as meaningful as big ones, as they represent realistic scenarios in domains such as chemistry and biology, where labeled data is costly to obtain and sometimes no where to be found and model robustness is essential. From the unsupervised category, we use Graph2Vec and NetLSD and from the supervised GIN. To ensure fairness, all methods are evaluated using a unified experimental pipeline with mini-

mal difference from GIN. The evaluation extends beyond standard classification and includes clustering behavior, computational efficiency and stability under random perturbations in the graphs structure.

The main contributions are summarized as follows:

- We present a comparative evaluation of unsupervised and supervised graph embedding methods on widely used small real world datasets,
- We analyze the trade-off between classification accuracy and computational cost, highlighting practical considerations for method selection,
- We investigate the clustering quality of the learned embeddings using standard clustering metrics, and
- We perform a stability analysis under structural permutations, assessing the robustness of different embedding paradigms

II. Related Work

III. Methods

This section describes the graph embedding methods evaluated in this work, along with the implementation theoritical details.

A. Graph2Vec

Starting with Graph2Vec [1], this is an unsupervised graph embedding method that learns a fixed length vector representation in a way analogous to how Doc2Vec learns embeddings for documents. So it's core idea is a graph treated like a document and the rooted subgraphs within it (meaning the local neighborhoods around nodes) are treated like words. The logic is that if two graphs share similar structural patterns their embeddings will be very close in the learned vector space.

The Graph2Vec's algorithm is shown below (from original paper):

Algorithm 1 Graph2Vec

Require: $\mathcal{G} = \{G_1, G_2, \ldots, G_{|\mathcal{G}|}\}$: set of graphs, where each $G_i = (N_i, E_i, \lambda_i)$
D: maximum WL depth
δ: embedding dimension
e: number of epochs
α: learning rate
Ensure: Graph embedding matrix $\Phi \in \mathbb{R}^{|\mathcal{G}| \times \delta}$
Initialize $\Phi \sim \mathcal{U}(-0.5/\delta, 0.5/\delta)$
for *epoch* $= 1$ to e **do**
    Shuffle $\mathcal{G}$
    for each graph $G_i \in \mathcal{G}$ **do**
        for each node $n \in N_i$ **do**
            for $d = 0$ to D **do**
                $sg \leftarrow$ GetWLSubgraph(n, G_i, d)
                Update Φ_i using Skip-Gram with learning rate α
            end for
        end for
    end for
end for
return Φ

Algorithm 2 GetWLSubgraph

Require: n: root node
$G = (N, E, \lambda)$: input graph
d: WL iteration depth
Ensure: Rooted subgraph label sg
$sg \leftarrow \lambda(n)$
for $i = 1$ to d **do**
    Collect multiset of neighbor labels of n
    Concatenate and hash labels
    Update sg
end for
return sg

So analytically, Graph2Vec first applies the Weisfeiler-Lehman (WL) relabeling process in order to extract rooted subgraphs around each node up to a specified depth D (Algorithm 2). Then, it creates a vocabulary called *bag of subgraphs* containing all rooted subgraphs. This vocabulary is formed by collecting the multiset of rooted subgraphs obtained across WL iterations for all graphs. And finally, it employs a skipgram model to learn graph embeddings by maximizing the likelihood of observing the subgraphs within each graph (Algorithm 1).

B. NetLSD

NetLSD [2] represent a given graph based on the spectrum of the graph Laplacian capturing global stactural information. Its main idea is that every graph has a Laplacian matrix whose eigenvalues encode important structural information. Instead of directly using the raw eigenvalues, it summarizes them using diffusion process, specifically the Heat Kernel.

The NetLSD's algorithm is shown below (not from original paper):

Algorithm 3 NetLSD (Heat Trace Graph Signature)

Require: Graph $G = (V, E)$
Require: $T = \{t_1, \ldots, t_K\}$: set of time scales (log-spaced)
Require: $\mathcal{L}$: Laplacian type (*normalized* or *combinatorial*)
Require: m: number of eigenvalues to use (optional; $m = |V|$ for exact)
Ensure: Signature vector $s \in \mathbb{R}^K$ (optionally normalized)
Construct adjacency matrix A of G and degree matrix D
if $\mathcal{L}$ is normalized **then**
  $L \leftarrow I - D^{-1/2} A D^{-1/2}$
else
  $L \leftarrow D - A$
end if
Compute smallest m eigenvalues $\{\lambda_1, \ldots, \lambda_m\}$ of L
for $k \leftarrow 1$ to K **do**
  $s_k \leftarrow \sum_{i=1}^{m} \exp(-t_k \lambda_i)$ ▷ heat trace
end for
$s \leftarrow \log(s)$ ▷ optional but common (stabilizes scale)
$s \leftarrow \textsc{Normalize}(s)$ ▷ e.g., $s/\|s\|_2$ (optional)
return s

The key advantage of this method lies in its permutation invariance and scalability as we will explore and confirm later.. To improve computational efficiency, it approximates the Laplacian eigenvalue distribution using stochastic trace estimation and so it avoids full eigendecomposition. This hepls NetLSD to maintain stable performace even for large graphs.

C. GIN

The Graph Isomorphism Network (GIN) [3], known as a supervised graph neural network, is designed to maximize discriminative power among GNNs. GIN's node representation are updated by aggregating context from neighbor nodes. At each layer, node features are updated using a sum aggregation function followed by a multilayer perceptron (MLP), allowing the model to distinguish different graph structures effectively as shown in Algorithm 4. Node embeddings are then aggregated using a global pooling operation to produce a graph representation. This representation is then passed to a classifier and trained end-to-end using labeled data as shown in Algorithm 5.

In contrast to unsupervised methods, GIN learns task specific embeddings optimized directly for classification accuracy. While this leads to great performances, it requires labeled datasets (which is difficult to obtain) and needs higher computational cost because it's iterative training.

It's algorithm is shown below (not from original paper):

Algorithm 4 GIN Forward Pass (Graph Embedding)

Require: Graph $G = (V, E)$, node features $\{h_v^{(0)}\}_{v \in V}$
Require: L: number of GIN layers
Require: $\{\epsilon^{(k)}\}_{k=1}^{L}$: learnable or fixed scalars
Require: $\{\mathrm{MLP}^{(k)}\}_{k=1}^{L}$: layer-wise MLPs
Require: READOUT: permutation-invariant pooling (sum/mean/max)
Ensure: Graph embedding z_G
for $k \leftarrow 1$ to L **do**
  for all $v \in V$ **do**
    $m_v^{(k)} \leftarrow \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}$
    $h_v^{(k)} \leftarrow \mathrm{MLP}^{(k)}\Big((1 + \epsilon^{(k)})\, h_v^{(k-1)} + m_v^{(k)}\Big)$
  end for
end for
$z_G \leftarrow \mathrm{READOUT}\big(\{h_v^{(L)}\}_{v \in V}\big)$
return z_G

Algorithm 5 Training a GIN for Graph Classification

Require: Dataset $\mathcal{D} = \{(G_i, y_i)\}_{i=1}^{N}$
Require: L: number of layers, epochs E, learning rate α, batch size B
Require: Loss function $\ell(\cdot, \cdot)$ (e.g., cross-entropy)
Ensure: Trained parameters Θ of GIN + classifier
Initialize model parameters Θ
for *epoch* $\leftarrow 1$ to E **do**
  Shuffle $\mathcal{D}$
  for each mini-batch $\mathcal{B} \subset \mathcal{D}$ of size B **do**
    for all $(G, y) \in \mathcal{B}$ **do**
      $z_G \leftarrow \textsc{GINForward}(G)$ ▷ Alg. 4
      $\hat{y} \leftarrow \textsc{Classifier}(z_G)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(G,y) \in \mathcal{B}} \ell(\hat{y}, y)$
    Update $\Theta \leftarrow \Theta - \alpha \nabla_\Theta \mathcal{L}$ ▷ e.g., Adam
  end for
end for
return Θ

D. Implementation Details

We implemented Graph2Vec and NetLSD using the KarateClub library [11], while GIN was implemented using PyTorch Geometric [12]. For all methods, we experimented with embedding dimensions of 64, 128, and 256. We thought to use smaller dimensions to reduce computational cost and memory usage, but we observed that with higher dims, we get better results as we will see in the results section.

IV. Datasets

The experiments in this work are conducted on three widely used benchmark datasets from the TUDataset collection: MUTAG, ENZYMES, and IMDB-MULTI. These datasets are small but diverse since they cover chemistry,

biology and social networks. They are standard benchmarks used in the graph representation learning literature [1]–[3] for evaluating graph embedding and classification methods.

A. MUTAG

MUTAG [4] is a molecular graph dataset consisting of chemical compounds where each graph represents a molecule, nodes correspond to atoms and edges represent chemical bonds between nodes/atoms. The task is to classify molecules into two classes according to their mutagenic effect on a bacterium. The dataset contains a small number of graphs with discrete node labels representing atom types. Graphs in MUTAG are known to be relatively small and sparse, so it is usefull for fast experimentation.

B. ENZYMES

The ENZYMES [5] dataset is derived from protein tertiary structures and contains graphs representing enzymes. Nodes correspond to secondary structure elements like amino acids and edges indicate spatial proximity or interactions between residues. Each graph is labeled according to the enzyme's functional class and as a result we have a multi class classification task, specifically 6 enzyme classes.

C. IMDB-MULTI

MDB-MULTI [5] is a social network dataset constructed from movie collaboration data. Each graph represents the ego network of actors appearing in a movie, nodes represent actors and edges show the coappearance relationships in the same movie. Unlike molecular datasets, IMDB-MULTI does not provide node labels or attributes.

V. Experiments and Evaluation

A. Evaluation Protocol for Graph Classification

We evaluate the graph embeddings using the three TUDataset as we mentioned earlier. Methods Graph2Vec and NetLSD are proccesses under the same pipeline for the classification, while supervised GIN follows its own supervised training protocol for the classification task. Performance for this task is reported using Accuracy, F1 score and AUC with extra computational measurments such as training time for all methods and peak tracemalloc mb, rss before mb, rss after mb for GIN, meaning memory usage during training. In addition we study the influence of different embedding dimensionality evaluating on the different dim sizes as we mentioned in earlier section (64, 128 and 256).

B. Classification Using Unsupervised Embeddings

For each dataset and embedding dim we load each graph's embedding from CSV files. Each CSV contains one row per graph with a label columb and d embedding feature columns. For each configuration, meaning dataset and dimension, we split our embeddings into 80% train and 20% test.

Before training any classifier, embeddings are standardized using sklearn's StandardScaler. This step is critical when training to distance and margin based classifier, like SVM, MLP etc., and ensures a consistent scale across all features. Since Graph2Vec and NetLSD are unsupervised, their embeddings are evaluated by training standard classifiers on top of the learned graph vectors. We choose 5 different classifiers:

- Support Vector Machine (SVM) with RBF kernel and probability calibration enabled. for AUC computation,
- Multi layer Perceptron (MLP), whose hidden layer sizes are adapted based on embedding dimensionality. We chose for larger embeddings to use deeper networks,
- Random Forest Classifier with with the number of trees and maximum depth adapted to dataset size to mitigate overfitting on small datasets,
- Logistic Regression with multinomial option for multiclass datasets, and
- Gradient Boosting with moderate depth

In addition, we combine Random Forest, Gradient Boosting and SVM to observe if we'll get better classification results by ensembling different classifiers. We named this combination as *Voting Classifier*. All results are shown and discussed in section *VI*.

C. Supervised Classification

Unlike Graph2Vec and NetLSD, GIN is trained in a supervised manner. Our implementation uses PyTorch Geometric and follows the architecture and training protocol described in Algorithms 4 and 5. So in contrast with the unsupervised methods, we don't use 5 different classifiers on top of GIN embeddings, because GIN itself is already a classifier. We computed the graph embeddings by training GIN and in parallel we optimize the loss for the classifiication metrics. As will be shown in the experimental results, this supervised training allows GIN to outperform the unsupervised methods on graph classification tasks.

The GIN model consists of stacked GINConv layers, each parameterized by an MLP, followed by batch normalization, ReLU activation, dropout and a residual connection. This design helps the model to learn complex graph structures while avoiding overfitting. To further enhance representational capacity, we employ jumping Knowledge with concatenation allowing the final graph embedding to incorporate information from all intermediate layers.

To improve classification results we add several additional design choices and training options. First, we run our experiments enabling -gfeat option, which concatenates simple graph level statistics (like the number of nodes, edges and graph density) to the learned embedding. Second, we use additive global pooling (–pool add), which is known to provide stable performance across datasets. Thrid, we apply feature normalization (–norm_feats) after constructing node features and we

observed that the optimization behavior improved. Finally, we apply label smoothing during training with value 0.05 (–label_smoothing 0.05) in the cross entropy loss to reduce overconfidence and improve generalization on small datasets.

To further study the effect of model capacity and training configuration, we vary multiple GIN hyperparameters during experimentation. Like the unsupervised methods, we evaluate GIN with embedding dimensions of 64, 128 and 256. We, also, trained for 200, 300, 500 and 800 epochs, depending on the dataset, but it is right to mention that this was a little bit overkill and that would be better if we could stick to better difference 50-200 epochs. Having, also, early stopping made the different epochs less critical. We explore, also, two learning rates $1 \cdot 10^{-3}$ and $5 \cdot 10^{-4}$, which are commonly used for training. Dropout is applied with rates of 0.0 and 0.4 to assess the impact of regularization on small datasets. For IMDB-MULTI, which contains larger graphs but no node attributes, we use a batch size of 64 (the same for MUTAG, while ENZYMES's batxhsize is set to 32), while the number of GIN layers is set to 3, reflecting the shallower architecture required for social network graphs. The number of layers for MUTAG is set to 5 and for ENZYMES to 6.

This hyperparameter exploration allows us to analyze how architectural depth, embedding dimensionality and regularization affect both classification accuracy and the quality of the learned graph embeddings. The results of these experiments are presented and discussed in section *VI*.

D. Clustering Evaluation

E. Clustering for Unsupervised Embeddings

F. Clustering for Supervised Embeddings

G. Stability Analysis

Because real world applications involve noisy data, for example edges may be missing and data in general may be corrupted, it is important to evaluate the robustness of our graph embeddings to such perturbations. So, we compute a stability analysis by introducing random permutations to the graph structures and node features.

For each dataset, we generated perturbed versions by applying 2 types of permutations: First, we apply edge pertubation, removing 10% of existing edges and add the same amount of new random edges. And secondly, we shuffle node attributes by randomly swapping features between pairs of nodes within each graph.

After generating these new datasets, we recompute embeddings using Graph2Vec, NetLSD and GIN with the same scripts and configurations as before. This ensures that any observed changes in the embeddings are solely due to the introduced perturbations and so we can fairly and safely make our comparisons between original and perturbed embeddings. To quality how much the embeddings have changed, we use two complementary measures:

TABLE I
GIN CLASSIFICATION PERFORMANCE ON EMB1 DATASETS (EPOCHS $\in \{200, 300\}$). BEST CONFIGURATION PER EMBEDDING DIMENSION IS REPORTED.

Dataset	Dim	Epochs	Acc	F1
MUTAG	64	200	0.842	0.825
MUTAG	128	200	0.842	0.808
MUTAG	256	200	0.895	0.864
ENZYMES	64	300	0.433	0.428
ENZYMES	128	300	0.483	0.492
ENZYMES	256	300	0.617	0.615
IMDB-MULTI	64	200	0.500	0.496
IMDB-MULTI	128	200	0.500	0.486
IMDB-MULTI	256	200	0.507	0.478

1) Cosine Similarity: For each graph, we compute the cosine similarity between its original embedding and the perturbed embedding. When we receive values close to 1, then the embeddings are very similar to the original one.

$$\cos(z_i, z_i') = \frac{z_i^T \cdot z_i'}{||z_i||_2 \cdot ||z_i'||_2}$$

2) Euclidean Distancw (L2 norm):

$$||z_i - z_i'||_2$$

where smaller distance means higher similarity between original and perturbed embeddings.

For each dataset, method and config, we report mean, median, and standard deviation of cosine similarity and L2 distance and then we compare its method in order to obseved somehow any difference in the stabilization logic.

After computing the above stability analysis, we further evaluate the practical impact of graph perturbations by repeating the classification and clustering experiments on the perturbed datasets with the same scripts and configs as we mentioned in the previous sections. Every result is reported and discussed in section *VI*.

VI. RESULTS

For the graph classification tasks, we present the performance of each embedding method across the three TUDatasets with accuracy, F1-score and AUC metrics for each method, along with computational metrics such as training time and memory usage. The results for GIN are summarized in Table I. We preferered to show only the best test configuration per dataset x dim results and for the epochs 200 and 300 for the reason we mentioned in earlier section.

The results for Graph2Vec and NetLSD are shown in Tables II and III respectively.

REFERENCES

[1] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, and S. Saminathan, "graph2vec: Learning Distributed Representations of Graphs," *arXiv preprint arXiv:1707.05005*, 2017.

[2] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, "NetLSD: Hearing the Shape of a Graph," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*, 2018.

TABLE II

BEST GRAPH2VEC CLASSIFICATION RESULTS PER DATASET AND EMBEDDING DIMENSION. FOR EACH DATASET–DIMENSION PAIR, THE CLASSIFIER WITH THE HIGHEST TEST ACCURACY IS REPORTED.

Dataset	**Dim**	**Model**	**Acc**	**F1**
MUTAG	64	RF	0.711	0.656
MUTAG	128	RF	0.763	0.754
MUTAG	256	RF	0.737	0.731
ENZYMES	64	Ens	0.258	0.252
ENZYMES	128	RF	0.225	0.219
ENZYMES	256	RF	0.317	0.310
IMDB-MULTI	64	RF	0.437	0.435
IMDB-MULTI	128	GB	0.427	0.426
IMDB-MULTI	256	SVM	0.433	0.426

TABLE III

BEST NETLSD CLASSIFICATION RESULTS PER DATASET AND EMBEDDING DIMENSION. FOR EACH DATASET–DIMENSION PAIR, THE CLASSIFIER WITH THE HIGHEST TEST ACCURACY IS REPORTED.

Dataset	**Dim**	**Model**	**Acc**	**F1**
MUTAG	64	Ens	0.895	0.892
MUTAG	128	Ens	0.868	0.867
MUTAG	256	LR	0.868	0.867
ENZYMES	64	RF	0.300	0.302
ENZYMES	128	RF	0.308	0.312
ENZYMES	256	RF	0.325	0.324
IMDB-MULTI	64	SVM	0.480	0.477
IMDB-MULTI	128	RF	0.463	0.465
IMDB-MULTI	256	RF	0.457	0.456

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[4] A. K. Debnath, R. L. López de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure–activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991, doi: 10.1021/jm00106a046.

[5] R. A. Rossi and N. K. Ahmed, "The Network Data Repository with Interactive Graph Analytics and Visualization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

[6] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler–Lehman Graph Kernels," *Journal of Machine Learning Research*, vol. 12, pp. 2539–2561, 2011.

[7] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt, "Graph Kernels," *Journal of Machine Learning Research*, vol. 11, pp. 1201–1242, 2010.

[8] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[9] F. Errica, M. Podda, D. Bacciu, and A. Micheli, "A Fair Comparison of Graph Neural Networks for Graph Classification," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[10] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking Graph Neural Networks," *arXiv preprint arXiv:2003.00982*, 2020.

[11] B. Rozemberczki, O. Kiss, and R. Sarkar, "Karate Club: An API Oriented Open-Source Python Framework for Unsupervised Learning on Graphs," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, 2020.

[12] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.